STAR UI Implementation Primer

Vue 3 + TypeScript Progressive Web App

STAR Development Team

December 2025

Contents

1 Project Overview

1.1 Purpose

The `star-ui` is a Progressive Web App (PWA) for controlling and monitoring the STAR robot. It provides:

- **Real-time Dashboard** with telemetry visualization
- **Map View** with live SLAM visualization
- **Motor Control** via gRPC-Web for low-latency commands
- **Offline Support** with service workers and IndexedDB
- **Installable** as native-like app on mobile devices

1.2 Technology Stack

Component	Technology	Version
Framework	Vue 3	3.4.x
Language	TypeScript	5.x
Build Tool	Vite	5.x
State Management	Pinia	2.x
GraphQL Client	Apollo Client	3.x
gRPC Client	gRPC-Web	1.x
PWA	Workbox	7.x
Styling	Tailwind CSS	3.x

Table 1: Technology Stack

1.3 Architecture

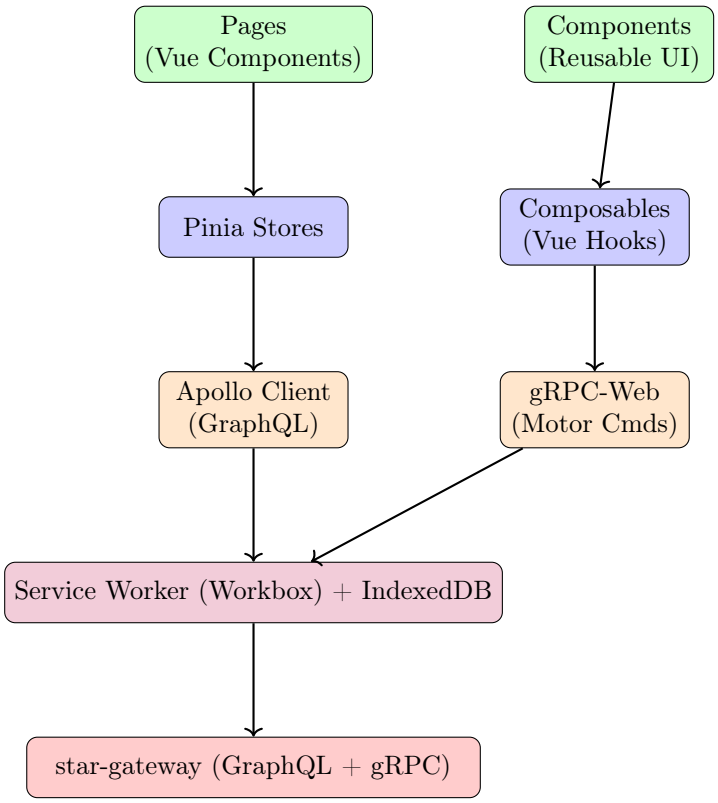

Figure 1: Frontend Architecture

2 Project Structure

```
star-ui/
|-- package.json
|-- vite.config.ts
|-- tsconfig.json
|-- tailwind.config.js
|-- index.html
|
|-- public/
|   |-- manifest.json              # PWA manifest
|   |-- icons/                     # App icons (192x192, 512x512)
|   +-- robots.txt
|
|-- src/
|   |-- main.ts                    # App entry point
|   |-- App.vue                    # Root component
|   |-- router/
|   |   +-- index.ts               # Vue Router config
|   |
|   |-- pages/
|   |   |-- DashboardPage.vue      # Main dashboard
```

```
|   |   |-- MappingPage.vue        # SLAM visualization
|   |   |-- ControlPage.vue        # Manual control
|   |   |-- SensorsPage.vue        # Sensor data
|   |   |-- SettingsPage.vue       # Configuration
|   |   +-- AnalyticsPage.vue      # Mission stats
|   |
|   |-- components/
|   |   |-- TelemetryCard.vue
|   |   |-- ImuVisualization.vue
|   |   |-- BatteryIndicator.vue
|   |   |-- MapCanvas.vue
|   |   |-- VirtualJoystick.vue
|   |   |-- ConnectionStatus.vue
|   |   +-- EventLog.vue
|   |
|   |-- composables/
|   |   |-- useRobotConnection.ts
|   |   |-- useTelemetry.ts
|   |   |-- useMotorControl.ts
|   |   |-- useNavigation.ts
|   |   +-- useOfflineStorage.ts
|   |
|   |-- stores/
|   |   |-- robotStore.ts          # Robot state
|   |   |-- telemetryStore.ts      # Telemetry data
|   |   |-- settingsStore.ts       # User settings
|   |   +-- navigationStore.ts     # Navigation state
|   |
|   |-- graphql/
|   |   |-- client.ts              # Apollo Client setup
|   |   |-- queries.ts             # GraphQL queries
|   |   |-- mutations.ts           # GraphQL mutations
|   |   +-- subscriptions.ts       # GraphQL subscriptions
|   |
|   |-- grpc/
|   |   |-- client.ts              # gRPC-Web client
|   |   +-- motor_control_pb.ts    # Generated protobuf
|   |
|   |-- services/
|   |   |-- offlineStorage.ts      # IndexedDB wrapper
|   |   +-- serviceWorker.ts       # SW registration
|   |
|   |-- types/
|   |   |-- telemetry.ts           # Type definitions
|   |   |-- navigation.ts
|   |   +-- settings.ts
|   |
|   +-- assets/
|       +-- styles/
|           +-- main.css           # Global styles
|
+-- workbox-config.js              # Service worker config
```

3 Implementation Tasks

3.1 Phase 1: Project Setup

3.1.1 Task 1.1: Initialize Vite Project

```
npm create vite@latest star-ui -- --template vue-ts
cd star-ui
npm install

# Install dependencies
npm install vue-router@4 pinia @apollo/client graphql
npm install @improbable-eng/grpc-web google-protobuf
npm install workbox-window idb
npm install -D tailwindcss postcss autoprefixer
npm install -D vite-plugin-pwa @types/google-protobuf
```

3.1.2 Task 1.2: Vite Configuration

Create `vite.config.ts`:

Listing 1: vite.config.ts

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { VitePWA } from 'vite-plugin-pwa'

export default defineConfig({
  plugins: [
    vue(),
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['favicon.ico', 'robots.txt', 'icons/*.png'],
      manifest: {
        name: 'STAR Robot Controller',
        short_name: 'STAR',
        description: 'Control and monitor the STAR robot',
        theme_color: '#1e40af',
        background_color: '#ffffff',
        display: 'standalone',
        icons: [
          {
            src: '/icons/icon-192x192.png',
            sizes: '192x192',
            type: 'image/png'
          },
          {
            src: '/icons/icon-512x512.png',
            sizes: '512x512',
            type: 'image/png'
          }
        ]
      },
      workbox: {
```

```
        globPatterns: ['**/*.{js,css,html,ico,png,svg}'],
        runtimeCaching: [
          {
            urlPattern: /^https:\/\/star-robot\.local\/graphql/,
            handler: 'NetworkFirst',
            options: {
              cacheName: 'graphql-cache',
              networkTimeoutSeconds: 3,
              cacheableResponse: {
                statuses: [0, 200]
              }
            }
          }
        ]
      }
    })
  ],
  server: {
    proxy: {
      '/graphql': 'http://star-robot.local:8080',
      '/graphql-ws': {
        target: 'ws://star-robot.local:8080',
        ws: true
      }
    }
  }
})
```

3.2 Phase 2: GraphQL Integration

3.2.1 Task 2.1: Apollo Client Setup

Create `src/graphql/client.ts`:

Listing 2: client.ts

```
import {
  ApolloClient,
  InMemoryCache,
  HttpLink,
  split
} from '@apollo/client/core'
import { GraphQLWsLink } from '@apollo/client/link/subscriptions'
import { createClient } from 'graphql-ws'
import { getMainDefinition } from '@apollo/client/utilities'

const httpLink = new HttpLink({
  uri: 'http://star-robot.local:8080/graphql'
})

const wsLink = new GraphQLWsLink(
  createClient({
    url: 'ws://star-robot.local:8080/graphql-ws'
  })
```

```
)

const splitLink = split(
  ({ query }) => {
    const definition = getMainDefinition(query)
    return (
      definition.kind === 'OperationDefinition' &&
      definition.operation === 'subscription'
    )
  },
  wsLink,
  httpLink
)

export const apolloClient = new ApolloClient({
  link: splitLink,
  cache: new InMemoryCache(),
  defaultOptions: {
    watchQuery: {
      fetchPolicy: 'cache-and-network'
    }
  }
})
```

3.2.2 Task 2.2: GraphQL Operations

Create `src/graphql/queries.ts`:

Listing 3: queries.ts

```
import { gql } from '@apollo/client/core'

export const GET_TELEMETRY = gql`
  query GetTelemetry {
    telemetry {
      battery
      temperature
      cpuUsage
      wifiSignal
      motorLoad
      timestamp
      imu {
        pitch
        roll
        yaw
        velocity
      }
    }
  }
`

export const GET_SYSTEM_STATUS = gql`
  query GetSystemStatus {
    systemStatus {
```

```
      connectionStatus
      mode
      esp32Connected
      lidarConnected
      rosConnected
    }
  }
`

export const GET_NAVIGATION_STATE = gql`
  query GetNavigationState {
    navigationState {
      currentPosition { x y heading }
      waypoints { id x y label }
      isNavigating
    }
  }
`
```

Create `src/graphql/subscriptions.ts`:

Listing 4: subscriptions.ts

```
import { gql } from '@apollo/client/core'

export const TELEMETRY_SUBSCRIPTION = gql`
  subscription TelemetryStream {
    telemetryStream {
      battery
      temperature
      cpuUsage
      timestamp
      imu {
        pitch
        roll
        yaw
        velocity
      }
    }
  }
`

export const LIDAR_SUBSCRIPTION = gql`
  subscription LidarScan {
    lidarScan {
      points { angle distance intensity }
      timestamp
    }
  }
`

export const EVENT_LOG_SUBSCRIPTION = gql`
  subscription EventLog {
    eventLog {
      timestamp
```

```
      message
      type
    }
  }
`
```

3.3 Phase 3: gRPC-Web Integration

3.3.1 Task 3.1: Generate Protobuf Types

```
# Install protoc and grpc-web plugin
npm install -g grpc-tools grpc_tools_node_protoc_ts

# Generate TypeScript from proto
protoc -I=../star-proto \
    --js_out=import_style=commonjs:./src/grpc \
    --grpc-web_out=import_style=typescript,mode=grpcweb:./src/grpc \
    ../star-proto/star/motor_control.proto
```

3.3.2 Task 3.2: gRPC Client

Create `src/grpc/client.ts`:

Listing 5: grpc/client.ts

```
import { MotorControlClient } from './MotorControlServiceClientPb'
import { VelocityCommand, EmptyRequest } from './motor_control_pb'

const GRPC_URL = 'http://star-robot.local:8090'

export const motorClient = new MotorControlClient(GRPC_URL)

export async function setVelocity(left: number, right: number) {
  const request = new VelocityCommand()
  request.setLeftVelocity(left)
  request.setRightVelocity(right)
  request.setTimestamp(Date.now())

  return new Promise((resolve, reject) => {
    motorClient.setVelocity(request, {}, (err, response) => {
      if (err) reject(err)
      else resolve(response.toObject())
    })
  })
}

export async function emergencyStop() {
  const request = new EmptyRequest()
  return new Promise((resolve, reject) => {
    motorClient.emergencyStop(request, {}, (err, response) => {
      if (err) reject(err)
      else resolve(response.toObject())
    })
```

```
  })
}
```

3.4 Phase 4: Composables

3.4.1 Task 4.1: Motor Control Composable

Create `src/composables/useMotorControl.ts`:

Listing 6: useMotorControl.ts

```
import { ref, computed } from 'vue'
import { setVelocity, emergencyStop } from '@/grpc/client'

export function useMotorControl() {
  const leftVelocity = ref(0)
  const rightVelocity = ref(0)
  const maxSpeed = ref(0.5)
  const isEmergencyStopped = ref(false)
  const lastCommandLatency = ref(0)

  const speed = computed(() =>
    (leftVelocity.value + rightVelocity.value) / 2
  )

  async function sendVelocity(left: number, right: number) {
    if (isEmergencyStopped.value) return

    const clampedLeft = Math.max(-maxSpeed.value, Math.min(maxSpeed.value,
        left))
    const clampedRight = Math.max(-maxSpeed.value, Math.min(maxSpeed.value
       , right))

    try {
      const start = performance.now()
      const response = await setVelocity(clampedLeft, clampedRight)
      lastCommandLatency.value = performance.now() - start

      leftVelocity.value = clampedLeft
      rightVelocity.value = clampedRight
    } catch (error) {
      console.error('Failed to send velocity:', error)
    }
  }

  async function stop() {
    await sendVelocity(0, 0)
  }

  async function triggerEmergencyStop() {
    try {
      await emergencyStop()
      isEmergencyStopped.value = true
      leftVelocity.value = 0
```

```
      rightVelocity.value = 0
    } catch (error) {
      console.error('Emergency stop failed:', error)
    }
  }

  function resetEmergencyStop() {
    isEmergencyStopped.value = false
  }

  return {
    leftVelocity,
    rightVelocity,
    maxSpeed,
    speed,
    isEmergencyStopped,
    lastCommandLatency,
    sendVelocity,
    stop,
    triggerEmergencyStop,
    resetEmergencyStop
  }
}
```

3.4.2 Task 4.2: Telemetry Composable

Create `src/composables/useTelemetry.ts`:

Listing 7: useTelemetry.ts

```
import { ref, onMounted, onUnmounted } from 'vue'
import { useSubscription } from '@vue/apollo-composable'
import { TELEMETRY_SUBSCRIPTION } from '@/graphql/subscriptions'

export function useTelemetry() {
  const telemetry = ref({
    battery: 0,
    temperature: 0,
    cpuUsage: 0,
    wifiSignal: 0,
    imu: { pitch: 0, roll: 0, yaw: 0, velocity: 0 }
  })

  const { result, error } = useSubscription(TELEMETRY_SUBSCRIPTION)

  onMounted(() => {
    // Subscribe to telemetry updates
  })

  return {
    telemetry,
    error
  }
}
```

3.5 Phase 5: PWA Features

3.5.1 Task 5.1: Offline Storage

Create `src/services/offlineStorage.ts`:

Listing 8: offlineStorage.ts

```
import { openDB, DBSchema } from 'idb'

interface StarDB extends DBSchema {
  telemetryHistory: {
    key: number
    value: {
      id: number
      timestamp: string
      battery: number
      temperature: number
      cpuUsage: number
    }
    indexes: { 'by-timestamp': string }
  }
  settings: {
    key: string
    value: any
  }
}

const DB_NAME = 'star-robot-db'
const DB_VERSION = 1

export async function initDB() {
  return openDB<StarDB>(DB_NAME, DB_VERSION, {
    upgrade(db) {
      const telemetryStore = db.createObjectStore('telemetryHistory', {
        keyPath: 'id',
        autoIncrement: true
      })
      telemetryStore.createIndex('by-timestamp', 'timestamp')

      db.createObjectStore('settings', { keyPath: 'key' })
    }
  })
}

export async function saveTelemetry(data: any) {
  const db = await initDB()
  await db.add('telemetryHistory', {
    ...data,
    timestamp: new Date().toISOString()
  })
}

export async function getTelemetryHistory(limit = 100) {
  const db = await initDB()
```

```
  const all = await db.getAllFromIndex('telemetryHistory', 'by-timestamp')
  return all.slice(-limit)
}

export async function saveSetting(key: string, value: any) {
  const db = await initDB()
  await db.put('settings', { key, value })
}

export async function getSetting(key: string) {
  const db = await initDB()
  const result = await db.get('settings', key)
  return result?.value
}
```

4 UI Pages

4.1 Dashboard Page

Features:

- Real-time battery, temperature, WiFi signal indicators
- IMU 3D visualization (pitch, roll, yaw)
- Connection status badges
- Quick action buttons (E-Stop, Mode switch)
- Event log stream

4.2 Mapping Page

Features:

- Live occupancy grid from SLAM
- Robot position and heading indicator
- LiDAR scan overlay
- Waypoint markers (clickable to add)
- Path history visualization
- Map export (PNG, PDF, GeoJSON)

4.3 Control Page

Features:

- Virtual joystick (touch/mouse)
- WASD keyboard controls

- Speed presets (0.1, 0.3, 0.5 m/s)
- Emergency stop button (always visible)
- Command latency display
- Mode selector (Manual, Autonomous, Mapping)

4.4 Sensors Page

Features:

- LiDAR polar plot
- IMU orientation graphs
- Ultrasonic distance readings
- Encoder velocities
- Raw data table

5 Build and Deploy

5.1 Development

```
cd star-ui

# Install dependencies
npm install

# Run dev server with hot reload
npm run dev

# Access at http://localhost:5173
```

5.2 Production Build

```
# Build for production
npm run build

# Output in dist/ directory

# Preview production build
npm run preview
```

5.3 Deployment Options

1. **Static hosting on Pi5**: Copy `dist/` to nginx/Apache
2. **CDN**: Deploy to Vercel, Netlify, or Cloudflare Pages
3. **Embedded**: Include in star-gateway as static resources

```
# Deploy to Pi5 nginx
scp -r dist/* root@star-robot.local:/var/www/html/

# Or serve from star-gateway
cp -r dist/* ../star-gateway/src/main/resources/static/
```

6 Testing

6.1 Unit Tests

```
npm install -D vitest @vue/test-utils happy-dom
npm run test
```

6.2 E2E Tests

```
npm install -D cypress
npx cypress open
```

6.3 PWA Audit

Use Chrome DevTools Lighthouse to audit:

- PWA installability
- Offline functionality
- Performance
- Accessibility

7 References

- Vue 3 Documentation: https://vuejs.org/
- Vite PWA Plugin: https://vite-pwa-org.netlify.app/
- Apollo Client Vue: https://v4.apollo.vuejs.org/
- gRPC-Web: https://github.com/grpc/grpc-web
- Workbox: https://developers.google.com/web/tools/workbox
- STAR Software Architecture Primer: Section 11
- STAR FTP Section 17: User Interface